



ИМИТАЦИОННОЕ МОДЕЛИРОВАНИЕ

Лабораторная работа № 2

Динамические модели SIR и Лотки–Вольтерры

Куокконен Дарина Андреевна
НФИбд-01-23

Российский университет дружбы народов
имени Патриса Лумумбы · Москва · 2026

Две модели показывают разные типы динамики

Цель — исследовать пороговую эпидемическую динамику и циклическое взаимодействие популяций средствами Julia.

- реализовать правые части обеих систем ОДУ;
- определить пик SIR и влияние числа контактов;
- найти равновесие и период Лотки–Вольтерры;
- проверить физические инварианты;
- получить Julia, QMD, IPYNB, HTML, DOCX, PDF и PPTX;
- подтвердить реализацию автоматическими тестами.

В SIR порог задаёт репродуктивное число

$$dS/dt = -\beta cSI/N$$

$$dI/dt = \beta cSI/N - \gamma I$$

$$dR/dt = \gamma I$$

$$R_0 = \beta c/\gamma$$

$$R_e(t) = R_0 S(t)/N$$

$S + I + R = N$ сохраняется.

При $R_0 > 1$ инфекция сначала распространяется.

После пересечения $R_e = 1$ число инфицированных убывает.

$R_0 = 2$

базовый сценарий

17,5 дня

момент пика

Лотка–Вольтерра образует замкнутые орбиты

$$dx/dt = \alpha x - \beta xy$$

$$dy/dt = \delta xy - \gamma y$$

$$(x^*, y^*) = (\gamma/\delta, \alpha/\beta)$$

$$H = \delta x - \gamma \ln x + \beta y - \alpha \ln y$$

Равновесие: (30, 5).

Начальное состояние задаёт конкретную замкнутую орбиту.

Дрейф первого интеграла служит контролем точности интегрирования.

37,6889

оценка периода

$1,82 \cdot 10^{-11}$

дрейф H

Проект повторяет конвейер лабораторной № 1

```
Terminal - dakuokkonen
dakuokkonen@lab02:~/study$ cd labs/lab02/project
dakuokkonen@lab02:~/study/lab02$ julia --version
julia version 1.11.9
dakuokkonen@lab02:~/study/lab02$ quarto --version
1.10.18
dakuokkonen@lab02:~/study/lab02$ jupyter lab --version
4.4.7
```

**Literate.jl → Julia → Jupyter →
Quarto → Git**

Julia 1.11.9

DifferentialEquations.jl

DrWatson.jl

Plots.jl

IJulia и Quarto

Версии закреплены в
Manifest.toml.

Один модуль содержит обе математические модели

```
Terminal — dakuokkonen

1.10.18
dakuokkonen@lab02:~/study/lab02$ jupyter lab --version
4.4.7
dakuokkonen@lab02:~/study/lab02$ ls
Makefile Manifest.toml Project.toml data markdown notebooks plots scripts src test
dakuokkonen@lab02:~/study/lab02$ sed -n '1,80p' src/epidemic_ecology.jl
module EpidemicEcology

function sir!(du, u, p, t)
    susceptible, infected, recovered = u
    beta, contacts, gamma = p
    population = susceptible + infected + recovered
    incidence = beta * contacts * susceptible * infected / population
    du[1] = -incidence
    du[2] = incidence - gamma * infected
    du[3] = gamma * infected
end

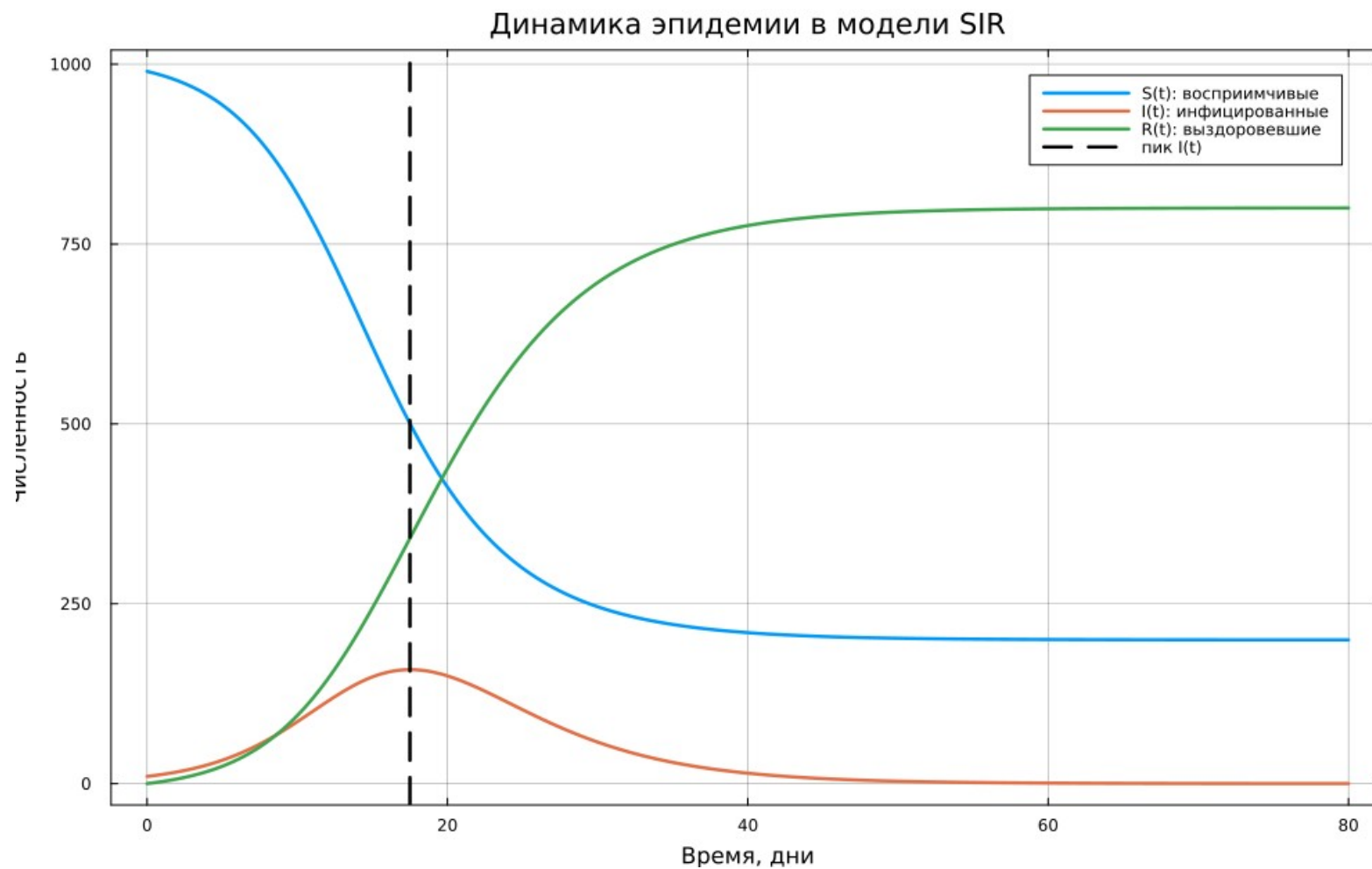
function lotka_volterra!(du, u, p, t)
    prey, predator = u
    alpha, beta, delta, gamma = p
    du[1] = alpha * prey - beta * prey * predator
    du[2] = delta * prey * predator - gamma * predator
end
```

Общие функции:

- sir!
- lotka_volterra!
- R_0 и $R_e(t)$
- равновесие
- первый интеграл

Сценарии и тесты
используют один источник
формул.

При $R_0 = 2$ пик возникает на 17,5-й день



158,4512

максимум $I(t)$

17,5

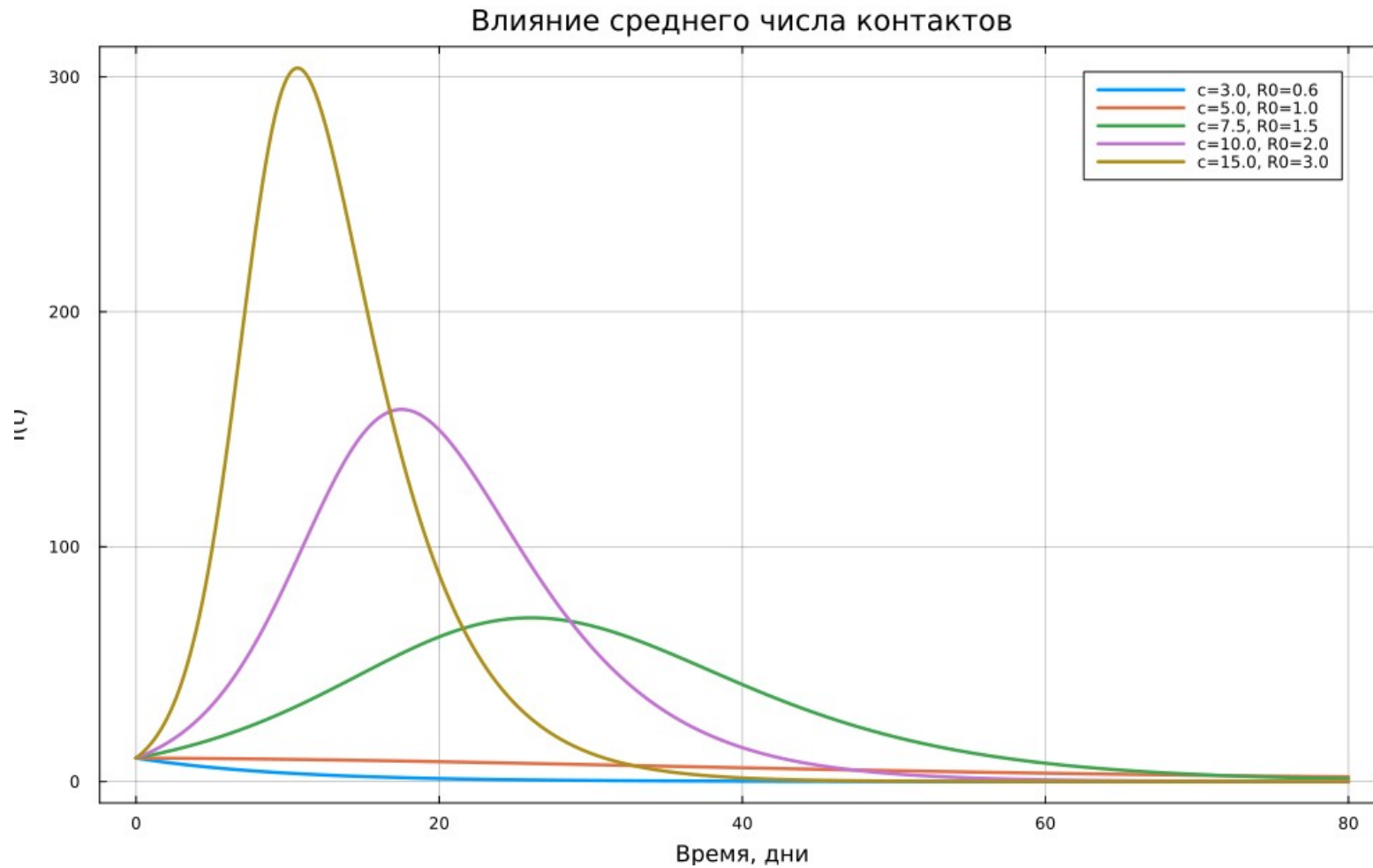
день пика

800,1424

$R(80)$

Дрейф $S + I + R < 8 \cdot 10^{-13}$

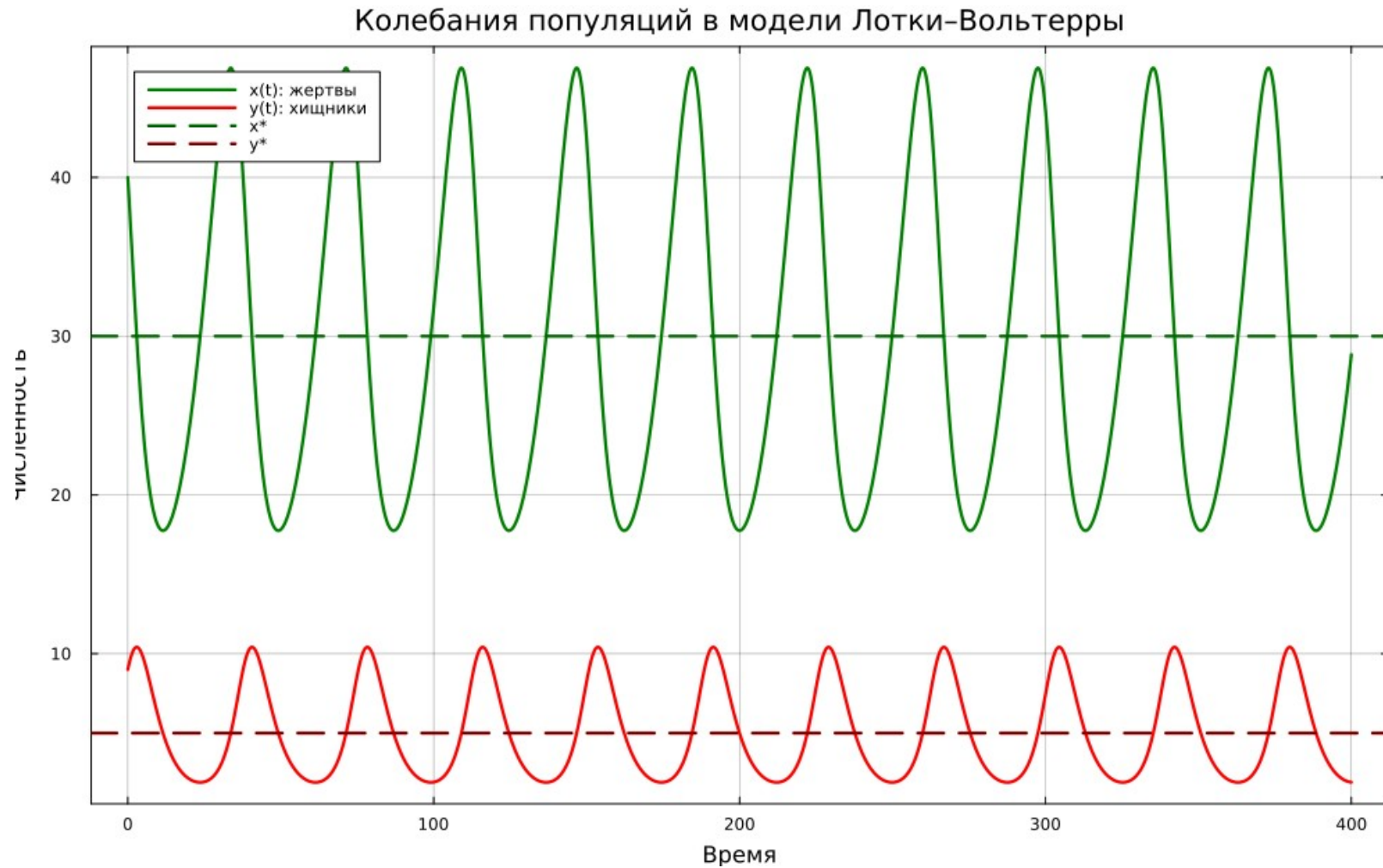
Число контактов управляет высотой и временем пика



c	R_0	I_{max}
3	0,6	10,0
5	1,0	10,0
7,5	1,5	69,7
10	2,0	158,5
15	3,0	303,8

Рост R_0 делает эпидемическую волну выше и сдвигает её к началу интервала.

Рост жертв опережает рост хищников



Равновесие: (30, 5)

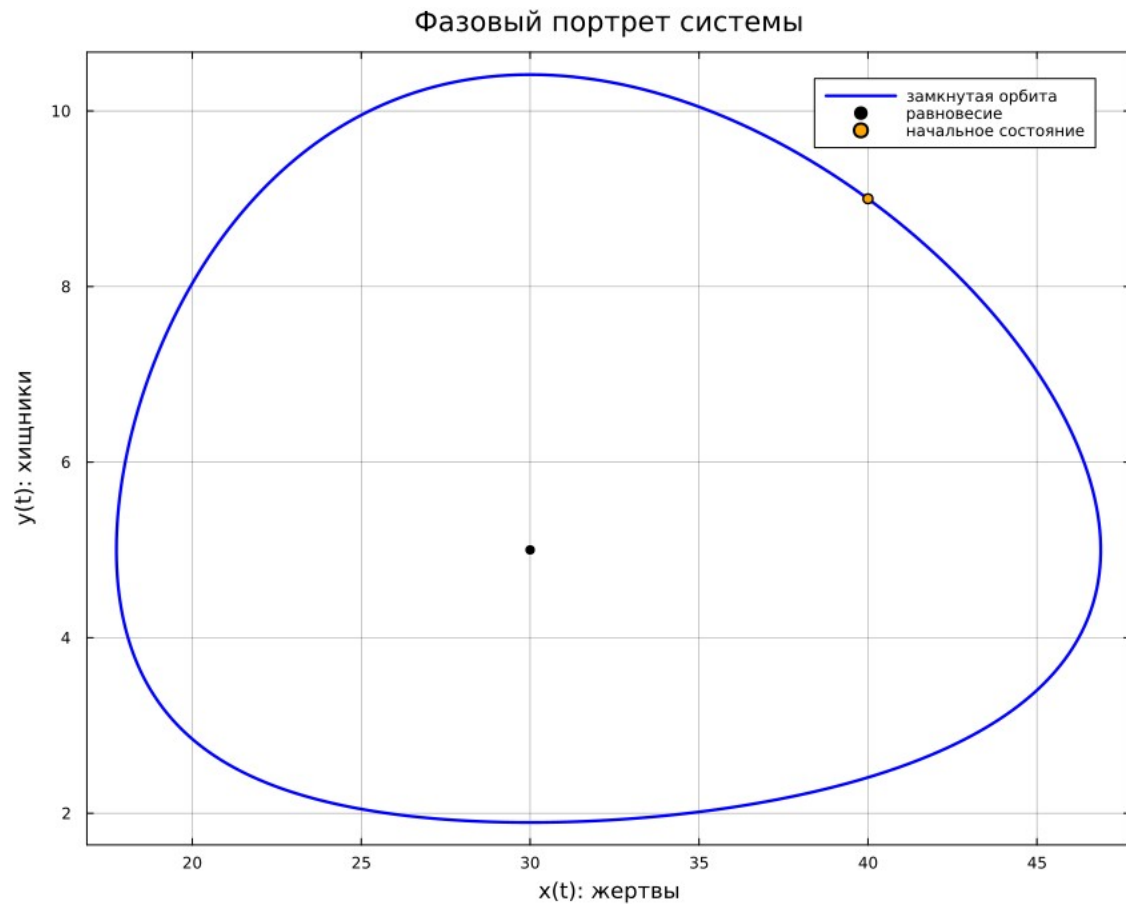
Период: 37,6889

Жертвы: 17,75–46,89

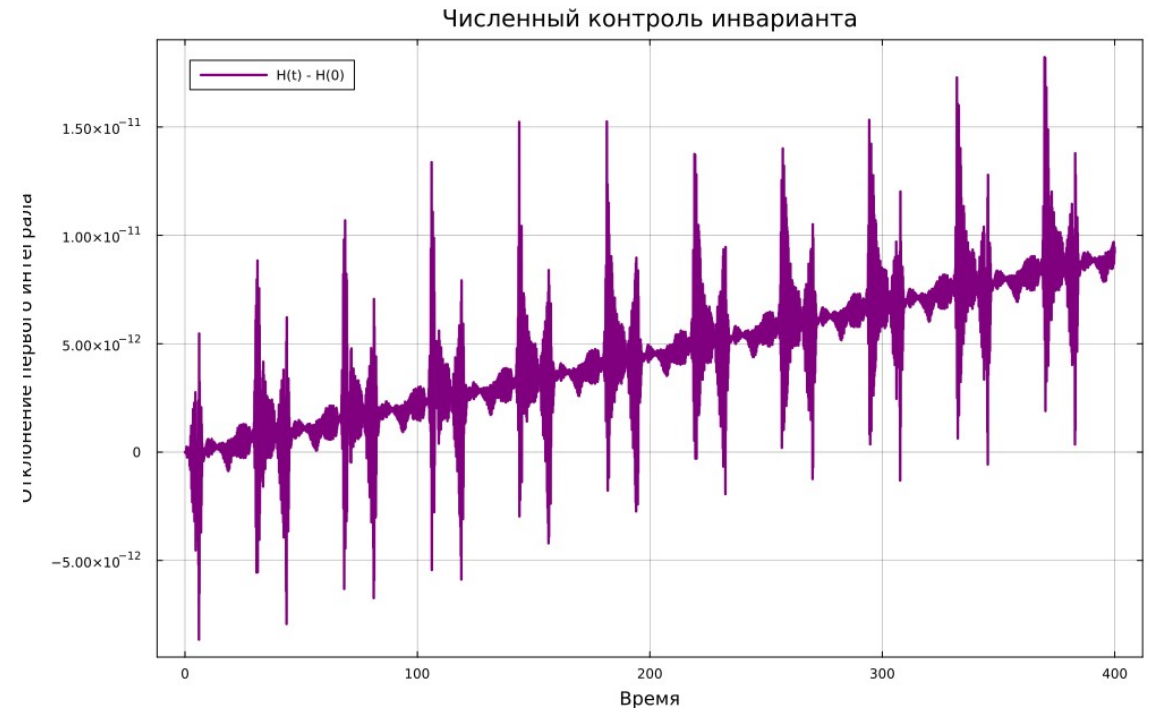
Хищники: 1,90–10,41

Максимум хищников следует за максимумом жертв — в системе возникает причинная задержка.

Фазовая орбита замкнута, а первый интеграл почти постоянен

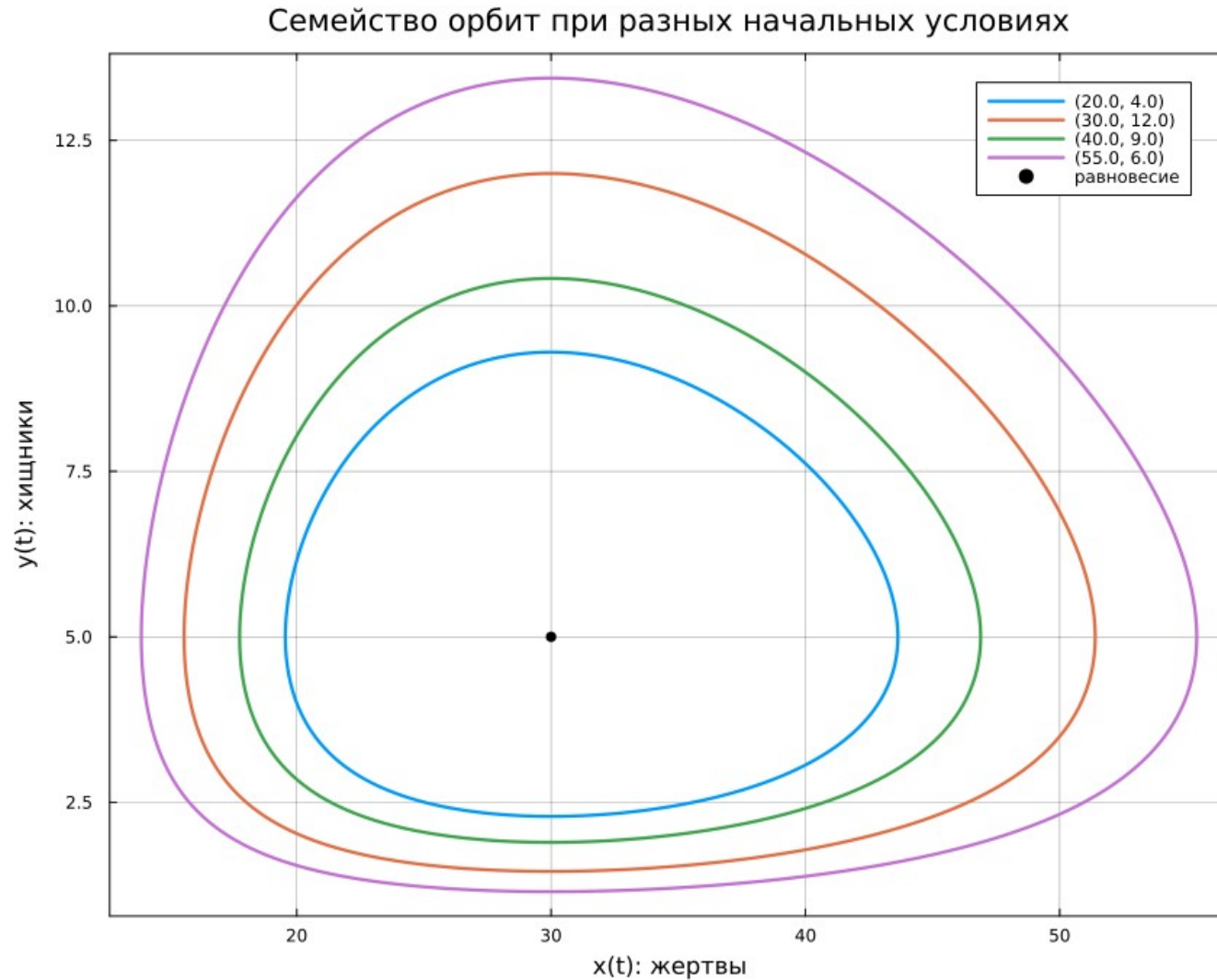


Орбита окружает точку (30, 5)



$\max |H(t) - H(0)| \approx 1,82 \cdot 10^{-11}$

Начальные условия меняют амплитуду, но не равновесие

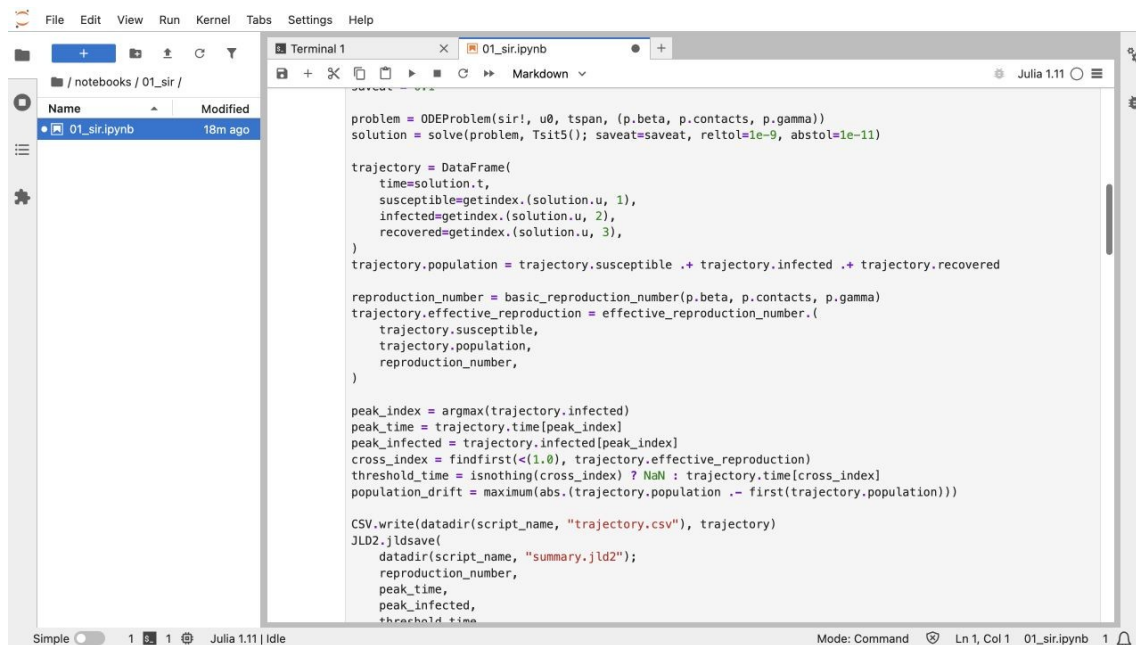


Исследованы
состояния:

(20, 4)
(30, 12)
(40, 9)
(55, 6)

Все орбиты окружают
одну точку (30, 5).

Juliyter используется только для интерактивного воспроизведения



```
problem = ODEProblem(sir!, u0, tspan, (p.beta, p.contacts, p.gamma))
solution = solve(problem, Tsit5(); saveat=saveat, reltol=1e-9, abstol=1e-11)

trajectory = DataFrame(
    time=solution.t,
    susceptible=getindex.(solution.u, 1),
    infected=getindex.(solution.u, 2),
    recovered=getindex.(solution.u, 3),
)
trajectory.population = trajectory.susceptible .+ trajectory.infected .+ trajectory.recovered

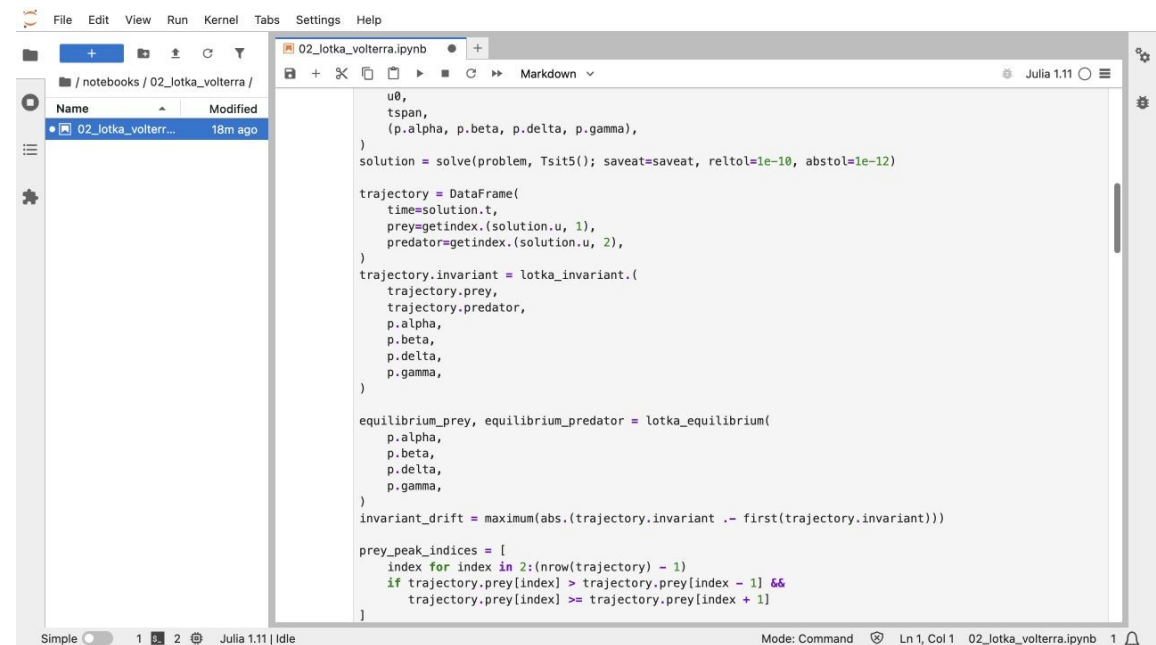
reproduction_number = basic_reproduction_number(p.beta, p.contacts, p.gamma)
trajectory.effective_reproduction = effective_reproduction_number.(
    trajectory.susceptible,
    trajectory.population,
    reproduction_number,
)

peak_index = argmax(trajectory.infected)
peak_time = trajectory.time[peak_index]
peak_infected = trajectory.infected[peak_index]
cross_index = findfirst(<(1.0), trajectory.effective_reproduction)
threshold_time = isnothing(cross_index) ? NaN : trajectory.time[cross_index]
population_drift = maximum(abs.(trajectory.population .- first(trajectory.population)))

CSV.write(datadir(script_name, "trajectory.csv"), trajectory)

JLD2.jldsave(
    datadir(script_name, "summary.jld2"),
    reproduction_number,
    peak_time,
    peak_infected,
    threshold_time,
```

Модель SIR



```
u0,
tspan,
(p.alpha, p.beta, p.delta, p.gamma),
)
solution = solve(problem, Tsit5(); saveat=saveat, reltol=1e-10, abstol=1e-12)

trajectory = DataFrame(
    time=solution.t,
    prey=getindex.(solution.u, 1),
    predator=getindex.(solution.u, 2),
)
trajectory.invariant = lotka_invariant.(
    trajectory.prey,
    trajectory.predator,
    p.alpha,
    p.beta,
    p.delta,
    p.gamma,
)

equilibrium_pre, equilibrium_predator = lotka_equilibrium(
    p.alpha,
    p.beta,
    p.delta,
    p.gamma,
)

invariant_drift = maximum(abs.(trajectory.invariant .- first(trajectory.invariant)))

prey_peak_indices = [
    index for index in 2:(nrow(trajectory) - 1)
    if trajectory.prey[index] > trajectory.prey[index - 1] &&
    trajectory.prey[index] >= trajectory.prey[index + 1]
]
```

Модель Лотки-Вольтерры

Десять тестов проверяют формулы и инварианты

```
Terminal - dakuokkonen

dakuokkonen@lab02:~/study/lab02$ make test
Test Summary: | Pass  Total  Time
SIR model     |    6     6  1.0s
Test Summary: | Pass  Total  Time
Lotka-Volterra model |    4     4  0.9s
```

6 проверок SIR

4 проверки Лотки–Вольтерры

- ✓ сохранение $S + I + R$
- ✓ положительность решений
- ✓ стационарная точка
- ✓ сохранение $H(x, y)$

Все тесты пройдены.

Весь конвейер повторяется пятью командами

```
$ make tangle
```

```
$ make run
```

```
$ make notebooks
```

```
$ make docs
```

```
$ make test
```

Результат:

- Julia-скрипты;
- CSV и графики;
- выполненные IPYNB;
- HTML-документация;
- DOCX и PDF отчёта;
- HTML и PPTX презентации;
- проверенная история Git.



Обе модели согласуются с теоретическими свойствами

- SIR показала пороговый переход и затухание после $R_e = 1$.
- Увеличение числа контактов повышает и ускоряет пик инфекции.
- Лотка–Вольтерра образует периодические замкнутые орбиты.
- Инварианты сохраняются с погрешностью 10^{-11} – 10^{-13} .
- Исходники, отчёт, презентация и тесты собираются воспроизводимо.